pypolymix

Stochastic surrogate models for uncertainty quantification

Pieterjan Robbe, Habib Najm, Cosmin Safta

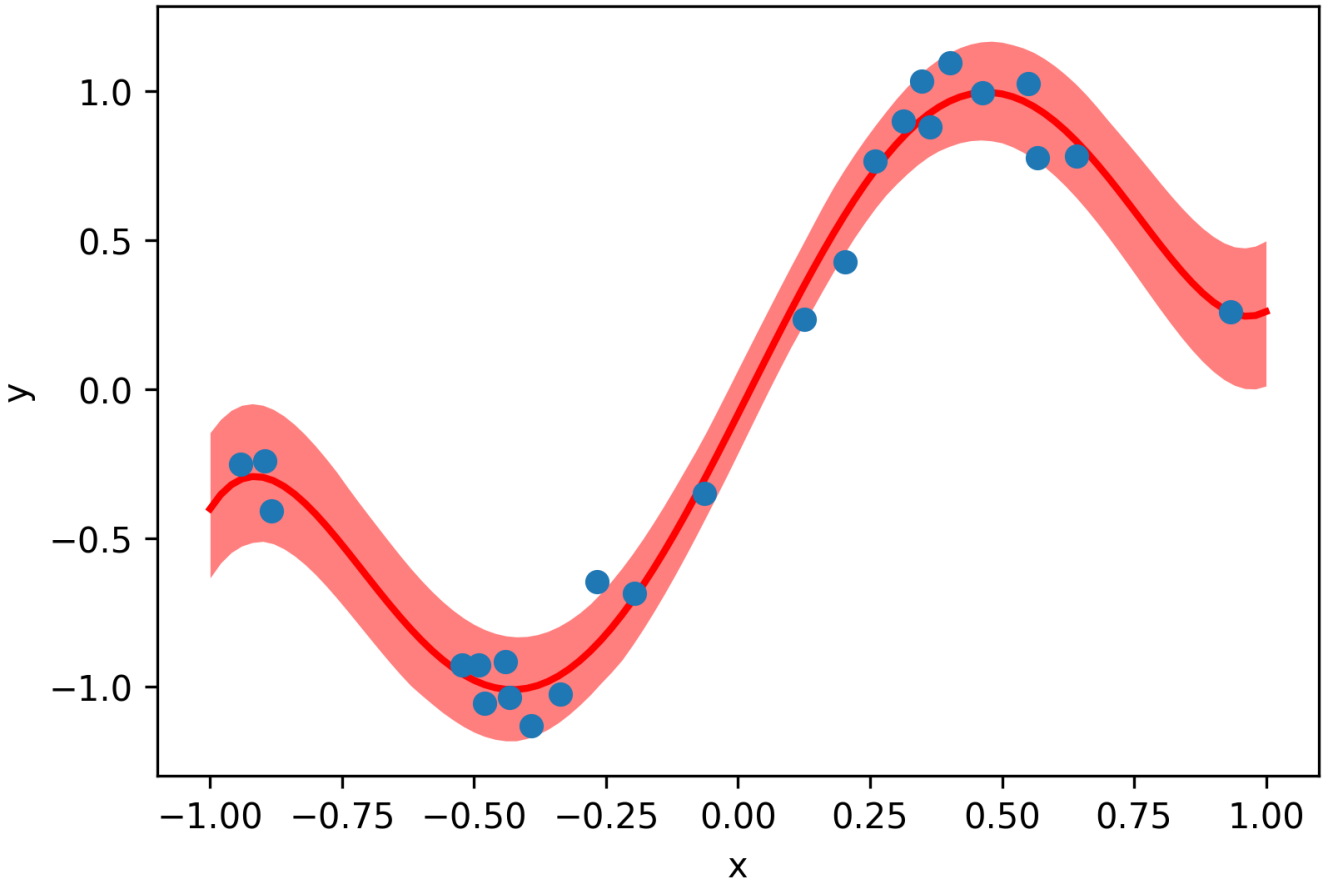
SNL

Table of contents

1. Welcome to pypolymix 3
   1.1 Key capabilities 3
   1.2 Quick start 3
2. Variational Inference 5
3. Package Reference 7
   3.1 API Reference 7
4. Examples 14
   4.1 Stochastic Polynomial Chaos Expansion¶ 14
   4.2 Stochastic Neural Network¶ 19
   4.3 Stochastic Mixture of Experts¶ 24
   4.4 Full-Covariance Polynomial Chaos¶ 31
   4.5 Priors Guide Ambiguous Line Fits¶ 37
5. Acknowledgements 42
   5.1 Acknowledgments 42

1. Welcome to pypolymix

`pypolymix` is a light-weight PyTorch companion for building **stochastic surrogate models**. Specify a deterministic surrogate architecture, attach variational parameter groups, and let `pypolymix` handle sampling, broadcasting, and regularization.



1.1 Key capabilities

- **Flexible surrogates:** Fully-connected networks, Polynomial Chaos expansions, Mixture-of-Experts, or any custom module that follows the `SurrogateModel` API.
- **Composable parameter groups:** Mix deterministic blocks and independent or full-covariance Gaussian families to approximate complex posteriors.
- **Prior-aware losses:** Every parameter group contributes a KL/cross-entropy term so you can tighten Bayesian objectives or regularize point estimates.

1.2 Quick start

The notebooks in `examples/sine_1d` walk through training stochastic surrogates on a noisy sine function. The core training loop looks like this:

```
import torch
from pypolymix import StochasticModel, parameter_groups, surrogate_models

# Synthetic training data
num_samples = 25
train_x = 2 * torch.rand(num_samples, 1) - 1
train_y = torch.sin(torch.pi * train_x) + 0.1 * torch.randn(train_x.shape)

# Polynomial Chaos Expansion surrogate
```

```
surrogate = surrogate_models.PolynomialChaosExpansion(num_inputs=1, num_outputs=1, degree=5)

# Variational parameter group: IID Gaussian
group = parameter_groups.IIDGaussianGroup("nn", surrogate.num_params())

# Stochastic model
model = StochasticModel(surrogate, group)

# Training loop
optimizer = torch.optim.Adam(model.parameters(), lr=5e-3)
for step in range(1000):
    preds = model(train_x, num_samples=32) # (samples, batch, outputs)
    data_loss = ((preds.mean(0) - train_y) ** 2).mean()
    loss = data_loss + 1e-3 * model.distribution_loss()

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

# Evaluate the model
model.eval()
with torch.no_grad():
    test_x = torch.linspace(-1, 1, 101).unsqueeze(1)
    test_y = model(test_x, num_samples=1000)

# Plot prediction
import matplotlib.pyplot as plt
_, ax = plt.subplots()
plot_x = test_x.squeeze(-1)
quantiles = torch.quantile(test_y, torch.tensor([0.33, 0.5, 0.66]), axis=0).squeeze(-1)
ax.fill_between(plot_x, quantiles[0], quantiles[-1], color="red", alpha=0.5, linewidth=0)
ax.plot(plot_x, quantiles[1], color="red", linewidth=2)
ax.scatter(train_x, train_y, zorder=99)
ax.set_xlabel("x")
ax.set_ylabel("y")
plt.tight_layout()
```

2. Variational Inference

Notations

- Observed data: $\mathcal{D} = \{(x_i, y_i) | i = 0, 1, \ldots d-1\}$
- Model parameters: $\mathbf{c} = \{c_0, c_1, \ldots, c_{n-1}\}$
- Likelihood & prior densities: $p(\mathcal{D}|\boldsymbol{c})$ & $p(\boldsymbol{c})$
- Posterior & approximate posterior densities: $p(\boldsymbol{c}|\mathcal{D})$ & $q_{\boldsymbol{\theta}}(\boldsymbol{c})$

In a Bayesian framework, the posterior distribution for the parameters $\boldsymbol{c}$ of a model M can be estimated as

$$p(\boldsymbol{c}|\mathcal{D}) \propto p(\mathcal{D}|\boldsymbol{c})p(\boldsymbol{c})$$

There are no closed-form solutions $p(\boldsymbol{c}|\mathcal{D})$ for generic models. Sampling the posterior distribution $p(\boldsymbol{c}|\mathcal{D})$ is typically hampered by high-dimensionality of $\boldsymbol{c}$ and the high computational cost to estimate the likelihood $p(\mathcal{D}|\boldsymbol{c})$ for a large number of model evaluations for example in Markov chain Monte Carlo schemes.

To work around this challenge, instead of sampling or evaluating $p(\boldsymbol{c}|\mathcal{D})$, we will *approximate* it with a simpler distribution $q_{\boldsymbol{\theta}}(\boldsymbol{c})$:

$$p(\boldsymbol{c}|\mathcal{D}) \approx q_{\boldsymbol{\theta}}(\boldsymbol{c})$$

where $\boldsymbol{\theta}$ are *variational parameters* to be optimized by minimizing the distance between the "true" and approximate distributions. The algorithm works as follows:

- Choose a tractable family of distributions $\mathcal{Q} = \{q_{\boldsymbol{\theta}} : \boldsymbol{\theta} \in \Theta\}$
- Use KL divergence as the measure of closeness, i.e.

$$\mathrm{KL}(q_{\boldsymbol{\theta}}(\boldsymbol{c}) \parallel p(\boldsymbol{c}|\mathcal{D})) = \int_{\boldsymbol{c}} q_{\boldsymbol{\theta}}(\boldsymbol{c}) \log \frac{q_{\boldsymbol{\theta}}(\boldsymbol{c})}{p(\boldsymbol{c}|\mathcal{D}))} \, d\boldsymbol{c}$$

to find $\boldsymbol{\theta}^* \in \Theta$ such that $q_{\boldsymbol{\theta}^*}(\boldsymbol{c})$ is closest to $p(\boldsymbol{c}|\mathcal{D})$ among all possible choices for $\boldsymbol{\theta}$.

The optimization algorithm uses the following **E**vidence **L**ower **BO**und (ELBO) expression for the loss function

$$\mathcal{L}(\boldsymbol{\theta}) = \mathbb{E}_q[\log p(\mathcal{D}|\boldsymbol{c})] - \mathrm{KL}(q_{\boldsymbol{\theta}}(\boldsymbol{c}) \parallel p(\boldsymbol{c})) + \mathrm{const.}$$

The current version of the library implements variational inference algorithms that assume Gaussian approximate posteriors, i.e. for $\mathcal{Q}$, as well as a Gausian likelihood. For the prior distribution $p(\boldsymbol{c})$, both uniform and Gaussian distributions are implemented.

For the case where both the approximate posterior and the prior are multivariate Gaussian distributions

$$q_{\boldsymbol{\theta}}(\boldsymbol{c}) = \mathcal{N}(\boldsymbol{c} \mid \boldsymbol{\mu}_{\boldsymbol{\theta}}, \boldsymbol{\Sigma}_{\boldsymbol{\theta}}), \;\; p(\boldsymbol{c}) = \mathcal{N}(\boldsymbol{c} \mid \boldsymbol{\mu}_p, \boldsymbol{\Sigma}_p),$$

the KL divergence can be computed analytically as

$$\mathrm{KL}(q_{\boldsymbol{\theta}}(\boldsymbol{c}) \parallel p(\boldsymbol{c})) = \frac{1}{2}\left[\mathrm{tr}(\boldsymbol{\Sigma}_p^{-1}\boldsymbol{\Sigma}_{\boldsymbol{\theta}}) + (\boldsymbol{\mu}_p - \boldsymbol{\mu}_{\boldsymbol{\theta}})^\top \boldsymbol{\Sigma}_p^{-1}(\boldsymbol{\mu}_p - \boldsymbol{\mu}_{\boldsymbol{\theta}}) - n + \log \frac{|\boldsymbol{\Sigma}_p|}{|\boldsymbol{\Sigma}_{\boldsymbol{\theta}}|}\right]$$

If the prior is uniform, the above expression reduces to

$$\mathrm{KL}(q_{\boldsymbol{\theta}}(\boldsymbol{c}) \parallel p(\boldsymbol{c})) = -\frac{1}{2}\log|\boldsymbol{\Sigma}_{\boldsymbol{\theta}}| + \mathrm{const.}$$

The first term on the right-hand side of the ELBO function

$$\mathbb{E}_q[\log p(\mathcal{D}|\boldsymbol{c})] \approx \frac{1}{N_s}\sum_{k=1}^{N_s}\log p(\mathcal{D}|\boldsymbol{c}_k)$$

with samples $\boldsymbol{c}_k$ drawn from $\mathcal{N}(\mu_{\boldsymbol{\theta}}, \Sigma_{\boldsymbol{\theta}})$. Using the reparametrization trick,

$$\boldsymbol{c} = \mu_{\boldsymbol{\theta}} + L\boldsymbol{\epsilon}$$

where $\boldsymbol{\epsilon} \propto \mathcal{N}(\mathbf{0}, \mathbb{I})$ and L is the lower-triangular matrix resulted from the Cholesky decomposition of $\Sigma_{\boldsymbol{\theta}}$, i.e. $\Sigma_{\boldsymbol{\theta}} = LL^T$. The expected log-likelihood expression above becomes

$$\mathbb{E}_q[\log p(\mathcal{D}|\boldsymbol{c})] = \mathbb{E}_{\boldsymbol{\epsilon}}[\log p(\mathcal{D}|\mu_{\boldsymbol{\theta}} + L\boldsymbol{\epsilon})] \approx \frac{1}{N_s}\sum_{k=1}^{N_s}\log p(\mathcal{D}|\mu_{\boldsymbol{\theta}} + L\boldsymbol{\epsilon}_k).$$

Gradients with respect to $\boldsymbol{\theta}$ components $\mu_{\boldsymbol{\theta}}$ and $\Sigma_{\boldsymbol{\theta}}$, are estimated through automatic differentiation tools if the model that depends of parameters $\boldsymbol{c}$ is differentiable.

3. Package Reference

3.1 API Reference

`pypolymix` splits stochastic surrogate modeling into three composable layers:

1. **Surrogate models** (`pypolymix.surrogate_models`): deterministic forward models that expect an input tensor and a tensor of parameters.
2. **Parameter groups** (`pypolymix.parameter_groups`): variational families with associated priors defined over blocks of surrogate parameters.
3. **Stochastic model** (`pypolymix.StochasticModel`): glues a surrogate to one or more parameter groups and exposes a familiar PyTorch `nn.Module` .

3.1.1 Stochastic Model

Wrap any surrogate in the `StochasticModel` framework and provide a list of parameter groups whose samples are concatenated before being fed to the surrogate.

```
import torch
from pypolymix import StochasticModel, parameter_groups, surrogate_models

surrogate = surrogate_models.NeuralNetwork(num_inputs=1, num_outputs=1)
group = parameter_groups.IIDGaussianGroup("nn", surrogate.num_params())
model = StochasticModel(surrogate, [group])

x = torch.linspace(-1, 1, 32).unsqueeze(-1)
y = model(x, num_samples=8)  # (8, 32, 1)
loss = y.mean() + model.distribution_loss()
```

Bases: `Module`

Wrap a deterministic surrogate model with sampled parameters.

Example

```
>>> from pypolymix import parameter_groups, surrogate_models
>>> surrogate = surrogate_models.NeuralNetwork(num_inputs=1, num_outputs=1)
>>> groups = [parameter_groups.IIDGaussianGroup("nn", surrogate.num_params())]
>>> model = StochasticModel(surrogate, groups)
>>> x = torch.linspace(-1, 1, 32).unsqueeze(-1)
>>> y = model(x, num_samples=4)  # (4, 32, 1)
```

Parameters:

- **`surrogate_model`** (`SurrogateModel`) – Base surrogate that consumes input `x` and sampled parameters.
- **`parameter_groups`** (`Union[ParameterGroup, Iterable[ParameterGroup]]`) – Parameter-group module or iterable of modules whose draws are concatenated to produce the full parameter vector expected by `surrogate_model` .

```
distribution_loss()
```

Return the sum of KL/cross-entropy terms provided by every parameter group.

```
forward(x, num_samples=1)
```

Evaluate the surrogate under randomly drawn parameters.

Parameters:

- **`x`** (`Tensor`) – Input tensor passed to the surrogate model.
- **`num_samples`** (`int` , default: `1`) – Number of parameter draws to average/evaluate over.

Example

Evaluate a function at `x` using 10 random draws of the parameters:

```
>>> y = model(x, num_samples=10)
>>> y.shape
torch.Size([10, x.shape[0], surrogate_model.num_outputs])
```

```
num_params()
```

Return the total number of scalar parameters managed across groups.

```
sample_parameters(num_samples=1)
```

Draw parameter samples from each group and concatenate them.

Parameters:

- **`num_samples`** (`int`, default: `1`) – Number of Monte Carlo samples per parameter group.

Returns:

- `Tensor` – Tensor with shape `(num_samples, surrogate_model.num_params())`.

3.1.2 Parameter Groups

Parameter groups describe how parameters are sampled and regularised. They can be mixed (e.g. deterministic biases and stochastic weights) by instantiating multiple groups and passing them to the same stochastic model.

DeterministicGroup

Use when you want point estimates for a parameter block while still leveraging the same interface as the stochastic groups.

Bases: `ParameterGroup`

Parameter group for deterministic inference.

Use this when optimisation should learn a single point estimate rather than sampling from a posterior approximation.

Example

```
>>> group = DeterministicGroup("weights", num_params=3)
>>> theta = group.sample_parameters(2)
>>> theta.shape
torch.Size([2, 3])
```

```
distribution_loss()
```

Return the negative log prior density evaluated at the current point.

```
sample_parameters(num_samples=1)
```

Return the same parameter vector repeated `num_samples` times.

```
variational_distribution()
```

Raises an error.

IIDGaussianGroup

Independent Normal posterior with per-parameter mean and (log) std that supports reparameterised sampling for variational inference.

Bases: `ParameterGroup`

I.i.d. Gaussian variational family: `q = Normal(mean, std)`.

Example

```
>>> group = IIDGaussianGroup("weights", num_params=4)
>>> samples = group.sample_parameters(16)
>>> samples.shape
torch.Size([16, 4])
```

```
std
```

Positive standard deviation parameter obtained via `exp`.

```
sample_parameters(num_samples=1)
```

Draw `n_samples` parameter vectors via `rsample` for reparameterization.

```
variational_distribution()
```

Return an independent normal distribution over all parameters.

GaussianGroup

Full-covariance Gaussian variational family parameterised by a Cholesky factor, useful when posterior correlations cannot be ignored.

Bases: `ParameterGroup`

Full-covariance Gaussian variational family with a learnable Cholesky factor.

This is useful when posterior correlations between parameters are important.

Example

```
>>> from pypolymix.parameter_groups import GaussianGroup
>>> group = GaussianGroup("weights", num_params=2)
>>> group.variational_distribution().rsample().shape
torch.Size([2])
```

```
sample_parameters(num_samples=1)
```

Draw `n_samples` reparameterized samples from the full-covariance Gaussian.

```
variational_distribution()
```

Return `torch.distributions.MultivariateNormal` with `scale_tril`.

3.1.3 Priors

All parameter groups accept a `Prior` object that creates a `torch.distributions.Distribution` on demand. Priors can therefore share learnable buffers or be re-used across groups.

IIDGaussianPrior

Bases: `Prior`

Independent Gaussian prior with per-parameter mean and standard deviation.

Example

prior = IIDGaussianPrior(mean=0.0, std=0.5) prior.distribution(torch.Size([4]), None, None).sample().shape torch.Size([4])

```
distribution(event_shape, device, dtype)
```

Return `Independent(N(mean, std), 1)` with broadcasted parameters.

GaussianPrior

Bases: `Prior`

Full-covariance Gaussian prior :math: `\mathcal{N}(\mu, \Sigma)`.

Users must provide either `covariance_matrix` or `scale_tril` when instantiating the prior; the other argument should be `None`.

Example

```
>>> mean = torch.zeros(2)
>>> cov = torch.eye(2)
>>> prior = GaussianPrior(mean, covariance_matrix=cov)
>>> isinstance(prior.distribution(torch.Size([2]), None, None), td.MultivariateNormal)
True
```

```
distribution(event_shape, device, dtype)
```

Validate tensor shapes and build a multivariate normal distribution.

LaplacePrior

Bases: `Prior`

IID Laplace prior that encourages sparsity.

Example

```
>>> prior = LaplacePrior(loc=0.0, scale=1e-1)
>>> prior.distribution(torch.Size([3]), None, None).sample().shape
torch.Size([3])
```

```
distribution(event_shape, device, dtype)
```

Return `Independent(Laplace(loc, scale), 1)` with broadcasted params.

3.1.4 Surrogate Models

Surrogates implement the deterministic mapping from `(x, params)` to outputs. They are ordinary PyTorch modules, but operate on batched parameter samples.

NeuralNetwork

Fully-connected MLP whose weights/biases are supplied dynamically via sampled parameters.

Bases: `SurrogateModel`

Neural network driven by sampled parameters.

Example

```
>>> surrogate = NeuralNetwork(num_inputs=2, num_outputs=1, width=8, depth=2)
>>> surrogate.num_params()
105
>>> params = torch.randn(3, surrogate.num_params())
>>> x = torch.randn(5, 2)
>>> surrogate(x, params).shape
torch.Size([3, 5, 1])
```

Parameters:

- **num_inputs** (int) – Dimensionality of x .
- **num_outputs** (int , default: 1) – Number of response dimensions.
- **width** (int , default: 16) – Hidden-layer width.
- **depth** (int , default: 1) – Number of hidden layers.
- **activation** (Callable , default: relu) – Callable applied after each hidden linear block.

```
forward(x, params)
```

Evaluate the neural network for multiple parameter samples in parallel.

Parameters:

- **x** (Tensor) – Tensor of shape (batch_size, num_inputs)
- **params** (Tensor) – Tensor of shape (num_samples, num_params)

Returns:

- **y** (Tensor) – Tensor of shape (num_samples, batch_size, num_outputs)

```
num_params()
```

Return the number of scalar parameters implied by the architecture.

PolynomialChaosExpansion

Legendre polynomial chaos expansion with configurable dimension, degree, and number of outputs.

Bases: SurrogateModel

Polynomial chaos expansion with Legendre basis.

Example

```
>>> surrogate = PolynomialChaosExpansion(num_inputs=1, degree=2)
>>> params = torch.randn(5, surrogate.num_params())
>>> x = torch.linspace(-1, 1, 20).unsqueeze(-1)
>>> surrogate(x, params).shape
torch.Size([5, 20, 1])
```

```
num_terms
```

Calculate the number of terms in the total order polynomial expansion.

```
forward(x, params)
```

Evaluate the polynomial chaos expansion.

Parameters:

- `x` (`Tensor`) – Sample points of shape (batch_size, num_inputs)
- `params` (`Tensor`) – Coefficient tensor of shape (num_samples, num_params)

Returns:

- `y` (`Tensor`) – PCE evaluations (num_samples, batch_size, num_outputs)

```
num_params()
```

Return `num_terms * num_outputs` .

Mixture Components

The mixture module contains both the gating network and the full Mixture-of-Experts surrogate, enabling scalable ensembles driven by sampled parameters.

GATINGNETWORK

Bases: `SurrogateModel`

Gating network that outputs mixture weights via softmax.

Example

```
>>> gating = GatingNetwork(num_inputs=1, num_experts=3, width=8)
>>> params = torch.randn(2, gating.num_params())
>>> x = torch.zeros(4, 1)
>>> gating(x, params).shape
torch.Size([2, 4, 3])
```

```
forward(x, params)
```

Compute mixture weights.

Parameters:

- `x` (`Tensor`) – (batch_size, num_inputs)
- `params` (`Tensor`) – (num_samples, num_params)

Returns:

- `Tensor` – Gating weights: (num_samples, batch_size, num_experts)

MIXTUREOFEXPERTS

Bases: `SurrogateModel`

Mixture of Experts surrogate model compatible with stochastic parameter sampling.

Example

```
>>> from pypolymix.surrogate_models import NeuralNetwork
>>> experts = [NeuralNetwork(num_inputs=1, num_outputs=1, width=4) for _ in range(2)]
>>> gating = GatingNetwork(num_inputs=1, num_experts=len(experts))
>>> moe = MixtureOfExperts(experts, gating)
>>> params = torch.randn(3, moe.num_params())
>>> x = torch.randn(5, 1)
>>> moe(x, params).shape
torch.Size([3, 5, 1])
```

Parameters:

- **`experts`** (`List[SurrogateModel]`) – List of expert surrogate models.
- **`gating_network`** (`SurrogateModel`) – A surrogate model returning mixture weights.

```
forward(x, params)
```

Vectorized forward pass through all experts and the gating network.

Parameters:

- **`x`** (`Tensor`) – (batch_size, num_inputs)
- **`params`** (`Tensor`) – (num_samples, total_params)

Returns:

- `Tensor` – Tensor of shape `(num_samples, batch_size, num_outputs)` containing
- `Tensor` – the expert ensemble prediction for each draw of `params` .

```
get_expert_outputs(x, params)
```

Evaluate each expert with the parameter slice assigned to it.

Returns:

- `Tensor` – Tensor with shape `(num_samples, num_experts, batch_size, num_outputs)` .

```
get_gating_weights(x, params)
```

Compute the mixture weights produced by the gating network.

```
num_params()
```

Total number of scalar parameters across all experts and gating network.

4. Examples

4.1 Stochastic Polynomial Chaos Expansion

This example builds a stochastic polynomial chaos expansion (PCE) surrogate for noisy samples of

$$y(x) = \sin(\pi x) + \varepsilon$$

with

$$\varepsilon \sim \mathcal{N}(0, 0.1^2)$$

We approximate the uncertain model response with a finite chaos basis,

$$\hat{y}(x, \boldsymbol{\xi}) = \sum_{k=0}^{p} c_k \Phi_k(x, \boldsymbol{\xi})$$

and infer an i.i.d. Gaussian distribution over the coefficients using `pypolymix`.

In [4…

```
# Import statements
import matplotlib.pyplot as plt
import torch

from pypolymix.parameter_groups import DeterministicGroup, IIDGaussianGroup
from pypolymix.surrogate_models import PolynomialChaosExpansion
from pypolymix import StochasticModel
```

In [4…

```
# Set random seed
_ = torch.manual_seed(1234)
```

We draw `num_samples = 25` scalar inputs uniformly in $[-1, 1]$ and evaluate $y_i = \sin(\pi x_i) + \varepsilon_i$ with Gaussian noise. This gives us a toy dataset whose curvature the PCE must learn.

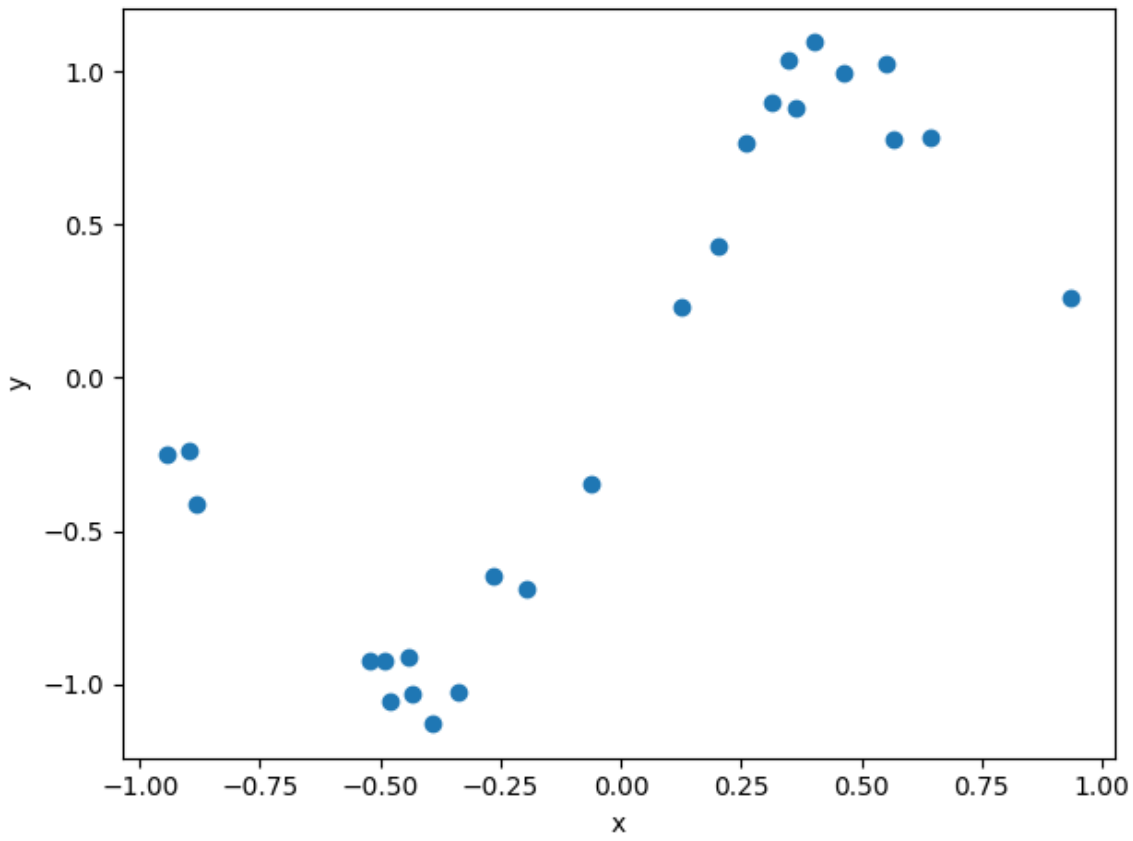
In [4…

```
# Generate synthetic data
num_samples = 25
X = 2 * torch.rand(num_samples, 1) - 1
Y = torch.sin(torch.pi * X) + 0.1 * torch.randn(X.shape)
```

A simple scatter plot reveals the wavy trend and the level of observational noise so that we can compare the eventual surrogate predictions against the ground truth samples.

In [4…

```
# Plot synthetic data
_, ax = plt.subplots()
ax.scatter(X, Y)
ax.set_xlabel("x")
ax.set_ylabel("y")
plt.tight_layout()
```



`PolynomialChaosExpansion(num_inputs=1, num_outputs=1, degree=5)` creates a one-dimensional chaos basis up to order five. We also print the resulting number of coefficients so we know how many stochastic parameters the training loop will handle.

In [4…

```
# Create a surrogate model
surrogate_model = PolynomialChaosExpansion(num_inputs=1, num_outputs=1, degree=5)
num_params = surrogate_model.num_params()
print(f"This model has {num_params} parameters")
```

```
This model has 6 parameters
```

`pypolymix` lets us divide the coefficient vector into interpretable blocks. Here we treat the first two coefficients as i.i.d. Gaussian (a stochastic prior) and the remaining four as deterministic. For the first two coefficients, `pypolymix` will infer a mean and (log of a) standard deviation internally.

In [5…

```
# Create parameter groups
parameter_groups = [
    IIDGaussianGroup("group0", 2),     # Assume first coefficient is iid Gaussian
    DeterministicGroup("group1", 4),   # All other coefficients are deterministic
]
```

`StochasticModel` couples the surrogate with the declared parameter groups.

In [5…

```
# Create stochastic model
model = StochasticModel(
    surrogate_model=surrogate_model, parameter_groups=parameter_groups
)
print(f"Created stochastic model with {model.num_params()} parameters")
```

```
Created stochastic model with 6 parameters
```

We set conventional hyperparameters (`lr`, `weight_decay`, `weight_factor`, `num_epochs`), construct an AdamW optimizer over the model parameters, choose mean-squared error as the data term, and add a `OneCycleLR` schedule to pace the learning rate.

In [5…

```python
# Training options
lr = 1e-3              # Learning rate
weight_decay = 1e-4    # Weight decay for AdamW
weight_factor = 5e-1   # Weight factor for distribution loss
num_epochs = 10000     # Number of epochs
num_samples = 100      # Number of parameter samples per epoch

# Optimizer: AdamW
optimizer = torch.optim.AdamW(model.parameters(), lr=lr, weight_decay=weight_decay)

# Loss
loss_fn = torch.nn.MSELoss(reduction="sum")

# Scheduler: OneCycleLR
scheduler = torch.optim.lr_scheduler.OneCycleLR(
    optimizer,
    max_lr=10 * lr,
    total_steps=num_epochs
)
```

During each epoch we draw 100 parameter realizations, push the training inputs through the surrogate, compute the data fit loss on the predictive mean, and add the distribution regularizer weighted by `weight_factor`.

In [5…

```python
# Train the stochastic model
for epoch in range(num_epochs):
    optimizer.zero_grad()

    # Evaluate parameters and model
    params = model.sample_parameters(num_samples=num_samples)
    Y_hat = surrogate_model(X, params)

    # Losses
    data_loss = loss_fn(Y_hat, Y.unsqueeze(0).expand_as(Y_hat)) / num_samples
    distribution_loss = model.distribution_loss()
    total_loss = data_loss + weight_factor * distribution_loss

    # Backprop + step
    total_loss.backward()
    optimizer.step()
    scheduler.step()

    # Logging
    if (epoch + 1) % 1000 == 0:
        current_lr = scheduler.get_last_lr()[0]
        print(
            f"Epoch {epoch + 1:5d} | "
            f"learning rate = {current_lr:.6f} | "
            f"data loss = {data_loss.item():.4f} | "
            f"distribution loss = {distribution_loss.item():.4f} | "
            f"total loss = {total_loss.item():.4f}"
        )
```

```
Epoch  1000 | learning rate = 0.002801 | data loss = 3.9538 |
distribution loss = 6.1787 | total loss = 7.0432
Epoch  2000 | learning rate = 0.007603 | data loss = 0.8597 |
distribution loss = 7.4630 | total loss = 4.5912
Epoch  3000 | learning rate = 0.010000 | data loss = 0.6857 |
distribution loss = 7.7149 | total loss = 4.5432
Epoch  4000 | learning rate = 0.009504 | data loss = 0.7021 |
distribution loss = 7.7233 | total loss = 4.5637
Epoch  5000 | learning rate = 0.008116 | data loss = 0.6816 |
distribution loss = 7.7523 | total loss = 4.5577
Epoch  6000 | learning rate = 0.006110 | data loss = 0.7567 |
distribution loss = 7.7501 | total loss = 4.6318
Epoch  7000 | learning rate = 0.003885 | data loss = 0.6301 |
distribution loss = 7.7676 | total loss = 4.5139
Epoch  8000 | learning rate = 0.001881 | data loss = 0.7846 |
distribution loss = 7.7322 | total loss = 4.6507
Epoch  9000 | learning rate = 0.000494 | data loss = 0.7777 |
distribution loss = 7.7341 | total loss = 4.6447
Epoch 10000 | learning rate = 0.000000 | data loss = 0.6668 |
distribution loss = 7.7338 | total loss = 4.5336
```

After training we switch to evaluation mode, create a dense set of test inputs, and draw multiple samples from the stochastic model to characterize predictive uncertainty across the domain.

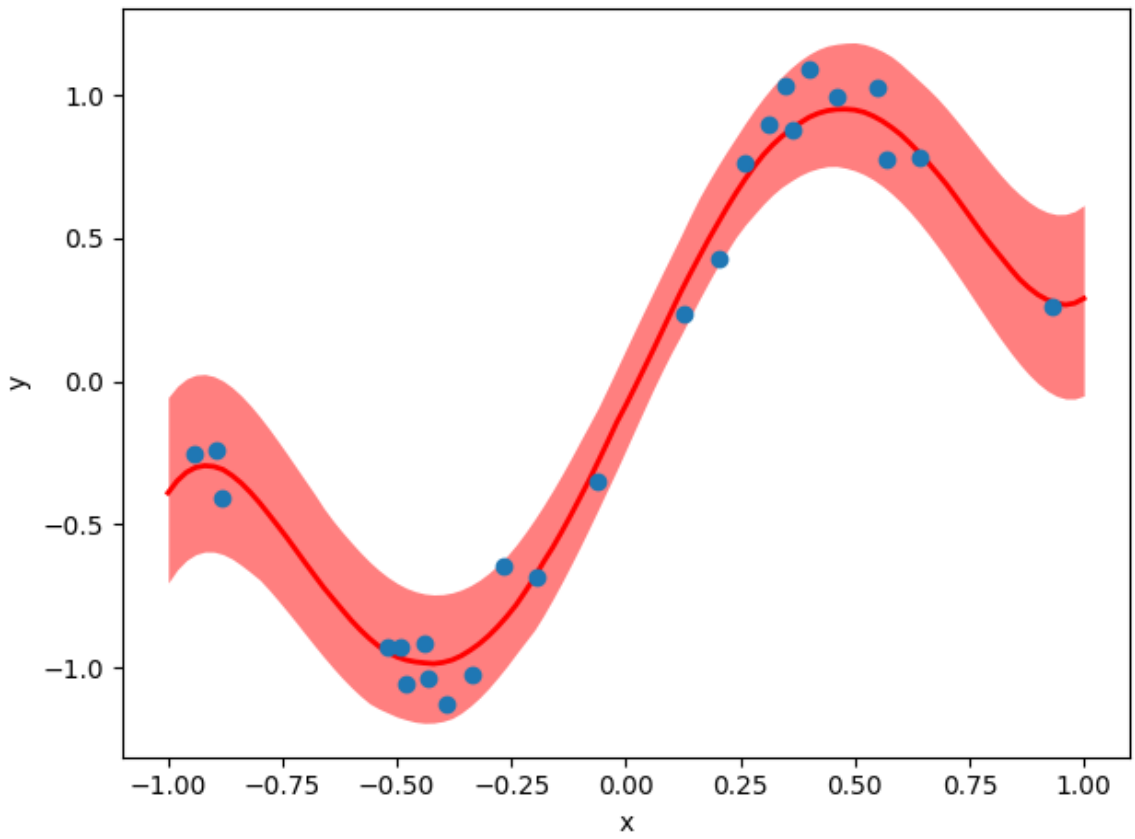
In [5…

```python
# Evaluate the model
model.eval()
with torch.no_grad():
    X_test = torch.linspace(-1, 1, 101).unsqueeze(1)
    Y_test = model(X_test, num_samples=100)
```

We summarize the sampled predictions by plotting the posterior mean and the central 90% interval via quantiles, then overlay the original data to visually assess how well the stochastic PCE tracks the signal.

In [5…

```python
# Plot prediction
_, ax = plt.subplots()
quantiles = torch.quantile(Y_test, torch.tensor([0.05, 0.5, 0.95]), axis=0)
X_plot = X_test.squeeze(-1)
quantiles = quantiles.squeeze(-1)
ax.fill_between(X_plot, quantiles[0], quantiles[-1], color="red", alpha=0.5, linewidth=0)
ax.plot(X_plot, quantiles[1], color="red", linewidth=2)
ax.scatter(X, Y, zorder=99)
ax.set_xlabel("x")
ax.set_ylabel("y")
plt.tight_layout()
```



In […

4.2 Stochastic Neural Network

Here we fit a stochastic neural network to two correlated outputs drawn from

$$y_1(x) = \sin(\pi x) + \varepsilon_1$$

and

$$y_2(x) = \cos(\pi x) + \varepsilon_2$$

with small Gaussian noise terms. The surrogate is a single-hidden-layer `tanh` network

$$\hat{\mathbf{y}}(x) = W_2 \sigma(W_1 x + b_1) + b_2$$

whose final-layer weights obey stochastic priors so we can sample functions.

In [1…

```
# Import statements
import matplotlib.pyplot as plt
import torch

from pypolymix.parameter_groups import IIDGaussianGroup, DeterministicGroup
from pypolymix.surrogate_models import NeuralNetwork
from pypolymix import StochasticModel
```

In [2…

```
# Set random seed
_ = torch.manual_seed(2048)
```

We draw `num_samples = 30` inputs in $[-1, 1]$ and evaluate $y_1(x)$ and $y_2(x)$ as sinusoidal functions plus Gaussian noise, stacking them into a two-column tensor `Y`.

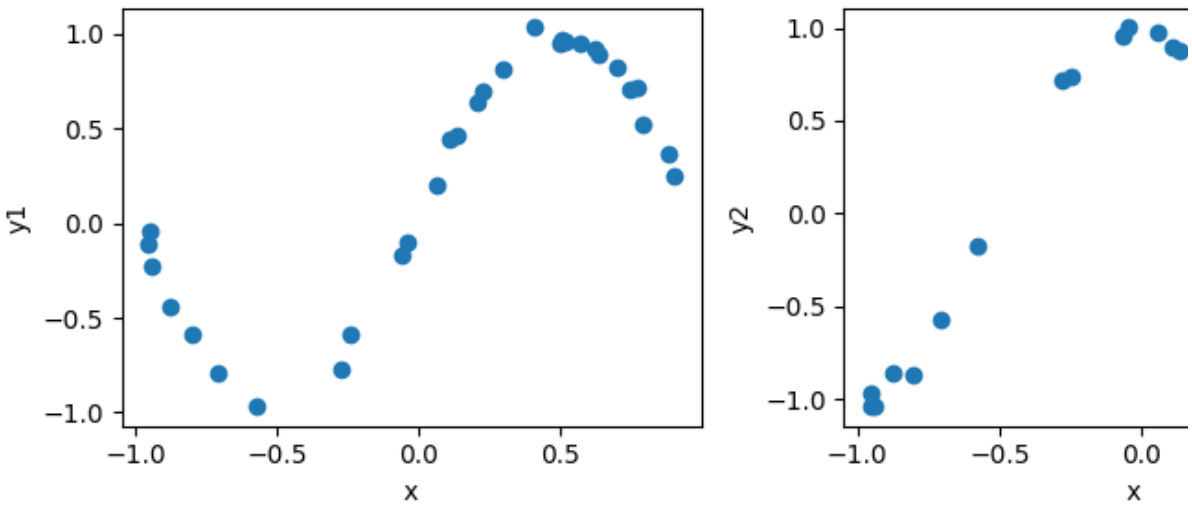
In [3…

```
# Generate synthetic data
num_samples = 30
X = 2 * torch.rand(num_samples, 1) - 1
y1 = torch.sin(torch.pi * X) + 0.05 * torch.randn(X.shape)
y2 = torch.cos(torch.pi * X) + 0.05 * torch.randn(X.shape)
Y = torch.hstack((y1, y2))
```

Two scatter plots (one per output) help visualize the phase shift between $\sin$ and $\cos$ so we know what structure the neural network must reproduce.

In [4…

```
# Plot synthetic data
_, axes = plt.subplots(1, 2, figsize=(8, 3))
for j, (y, ax) in enumerate(zip(Y.T, axes.flatten())):
    ax.scatter(X, y)
    ax.set_xlabel("x")
    ax.set_ylabel(f"y{j + 1}")
plt.tight_layout()
```



We configure a width-16, depth-1 tanh network and print how many trainable parameters it contains so we have a sense of model capacity.

In [5…

```
# Create a surrogate model
width = 16
depth = 1
surrogate_model = NeuralNetwork(num_inputs=1, num_outputs=2, width=width, depth=depth,
activation=torch.nn.functional.tanh)
num_params = surrogate_model.num_params()
print(f"This model has {num_params} parameters")
```

```
This model has 66 parameters
```

All weights/biases up through the hidden layer are treated as deterministic, while the output-layer weights and biases follow i.i.d. Gaussian priors. This split lets us capture uncertainty primarily in the mapping from hidden features to the outputs, similar to a Bayesian neural network in the "Bayesian in the last layer" configuration.

In [6…

```
# Create parameter groups
parameter_groups = [  # Input layer
    DeterministicGroup("input_layer_weights", width),
    DeterministicGroup("input_layer_biases", width),
]
for j in range(depth - 1):
    parameter_groups += [  # Hidden layers
        DeterministicGroup(f"layer{j + 1}_weights", width * width),
        DeterministicGroup(f"layer{j + 1}_biases", width),
    ]
parameter_groups += [  # Output layer
    IIDGaussianGroup("output_weights", 2 * width),
    IIDGaussianGroup("output_biases", 2),
]
```

The `StochasticModel` pairs the deterministic neural network with the grouped parameters so that sampling, log-prior penalties, and inference utilities become available.

In [7…

```
# Create stochastic model
model = StochasticModel(
    surrogate_model=surrogate_model, parameter_groups=parameter_groups
)
print(f"Created stochastic model with {model.num_params()} parameters")
```

```
Created stochastic model with 66 parameters
```

We pick learning-rate and weight-decay values, set a small `weight_factor` for the distribution loss, introduce a learnable log-precision `log_tau`, and build the AdamW optimizer plus `OneCycleLR` scheduler.

In [8…

```python
# Training options
lr = 1e-3              # Learning rate
weight_decay = 1e-4    # Weight decay for AdamW
weight_factor = 1e-2   # Weight factor for distribution loss
num_epochs = 10000     # Number of epochs
num_samples = 100      # Number of parameter samples per epoch

# Learnable global precision
log_tau = torch.nn.Parameter(torch.tensor(0.0))  # τ = exp(log_tau)

# Optimizer: AdamW
optimizer = torch.optim.AdamW(
    list(model.parameters()) + [log_tau], lr=lr, weight_decay=weight_decay
)

# Loss
loss_fn = torch.nn.MSELoss(reduction="sum")

# Scheduler: OneCycleLR
scheduler = torch.optim.lr_scheduler.OneCycleLR(
    optimizer,
    max_lr=10 * lr,
    total_steps=num_epochs
)
```

Each epoch samples 100 parameter realizations, evaluates the network, computes the precision-scaled MSE on the predictive mean, adds the distribution regularizer, and performs backpropagation followed by the optimizer/scheduler updates.

In [9…

```python
# Train the stochastic model
for epoch in range(num_epochs):
    optimizer.zero_grad()

    # Evaluate parameters and model
    params = model.sample_parameters(num_samples=100)
    Y_hat = surrogate_model(X, params)

    # Losses
    data_loss = loss_fn(Y_hat, Y.unsqueeze(0).expand_as(Y_hat)) / num_samples
    data_loss *= torch.exp(log_tau)
    distribution_loss = model.distribution_loss()
    total_loss = data_loss + weight_factor * distribution_loss

    # Backprop + step
    total_loss.backward()
    optimizer.step()
    scheduler.step()

    # Logging
    if (epoch + 1) % 1000 == 0:
        current_lr = scheduler.get_last_lr()[0]
        print(
            f"Epoch {epoch + 1:5d} | "
            f"learning rate = {current_lr:.6f} | "
            f"data loss = {data_loss.item():.4f} | "
            f"distribution loss = {distribution_loss.item():.4f} | "
            f"total loss = {total_loss.item():.4f}"
        )
```

```
Epoch  1000 | learning rate = 0.002801 | data loss = 33.4647 |
distribution loss = 79.6132 | total loss = 34.2608
Epoch  2000 | learning rate = 0.007603 | data loss = 3.9910 |
distribution loss = 89.9401 | total loss = 4.8904
Epoch  3000 | learning rate = 0.010000 | data loss = 1.1182 |
distribution loss = 98.5514 | total loss = 2.1037
Epoch  4000 | learning rate = 0.009504 | data loss = 0.4949 |
distribution loss = 103.0698 | total loss = 1.5256
Epoch  5000 | learning rate = 0.008116 | data loss = 0.2993 |
distribution loss = 104.3943 | total loss = 1.3432
Epoch  6000 | learning rate = 0.006110 | data loss = 0.2160 |
distribution loss = 104.0745 | total loss = 1.2567
Epoch  7000 | learning rate = 0.003885 | data loss = 0.1736 |
distribution loss = 102.8993 | total loss = 1.2026
Epoch  8000 | learning rate = 0.001881 | data loss = 0.1639 |
distribution loss = 101.5564 | total loss = 1.1795
Epoch  9000 | learning rate = 0.000494 | data loss = 0.1493 |
distribution loss = 100.6727 | total loss = 1.1560
Epoch 10000 | learning rate = 0.000000 | data loss = 0.1253 |
distribution loss = 100.4727 | total loss = 1.1300
```

After training we switch to eval mode, create evenly spaced test points, and ask the stochastic model for 1000 posterior samples so we can estimate predictive uncertainty.

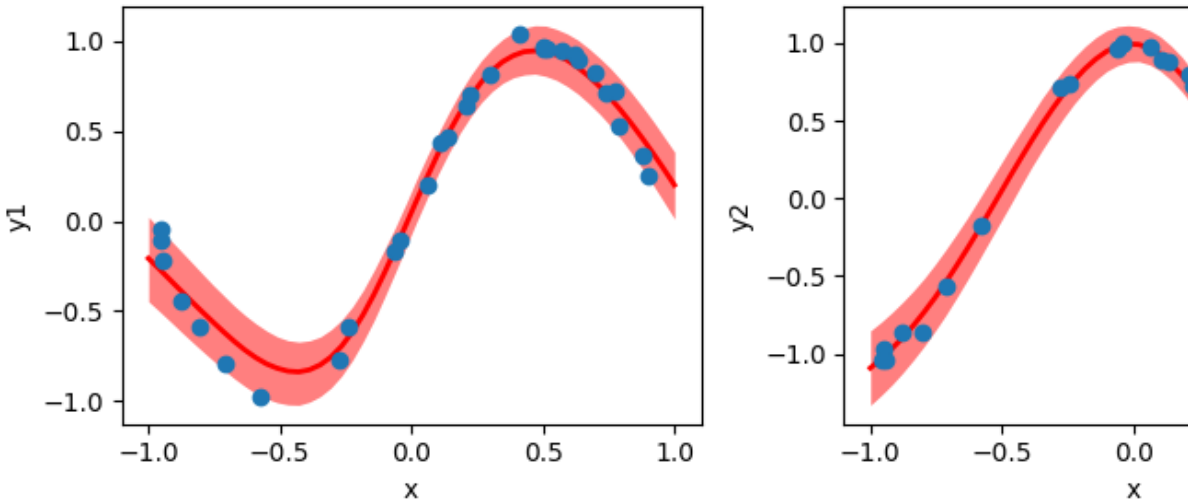
In [1…

```
# Evaluate the model
model.eval()
with torch.no_grad():
    X_test = torch.linspace(-1, 1, 50).unsqueeze(1)
    Y_test = model(X_test, num_samples=1000)
```

For each output dimension we show the mean prediction and the central 90% credible interval obtained from the samples, plus the observed data for reference.

In [1…

```
# Plot prediction
_, axes = plt.subplots(1, 2, figsize=(8, 3))
for j, (y, ax) in enumerate(zip(Y.T, axes.flatten())):
    x = X_test.squeeze(-1)
    q = torch.quantile(Y_test[:, :, j], torch.tensor([0.05, 0.5, 0.95]), axis=0)
    ax.fill_between(x, q[0], q[-1], color="red", alpha=0.5, linewidth=0)
    ax.plot(x, q[1], color="red", linewidth=2)
    ax.scatter(X, y, zorder=99)
    ax.set_xlabel("x")
    ax.set_ylabel(f"y{j + 1}")
plt.tight_layout()
```



In […

4.3 Stochastic Mixture of Experts

This notebook constructs a stochastic mixture-of-experts (MoE) model for observations of

$$y(x) = \sin(\pi x) + \operatorname{sign}(x) + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, 0.05^2)$$

The surrogate combines two polynomial chaos experts whose outputs are blended by a neural gating network so that the predictive mean is

$$\hat{y}(x) = \sum_{m=1}^{M} \pi_m(x) f_m(x)$$

with stochastic parameters in both experts and the gate.

In [1…

```python
# Import statements
import matplotlib.pyplot as plt
import torch

from pypolymix.parameter_groups import DeterministicGroup, IIDGaussianGroup
from pypolymix.surrogate_models import PolynomialChaosExpansion, MixtureOfExperts,
GatingNetwork
from pypolymix import StochasticModel
```

In [2…

```python
# Set random seed
_ = torch.manual_seed(1234)
```

We sample `num_samples = 50` inputs in $[-1, 1]$ and compute targets using $y_i = \sin(\pi x_i) + \operatorname{sign}(x_i) + \varepsilon_i$. The discontinuity at $x=0$ motivates the use of multiple experts.

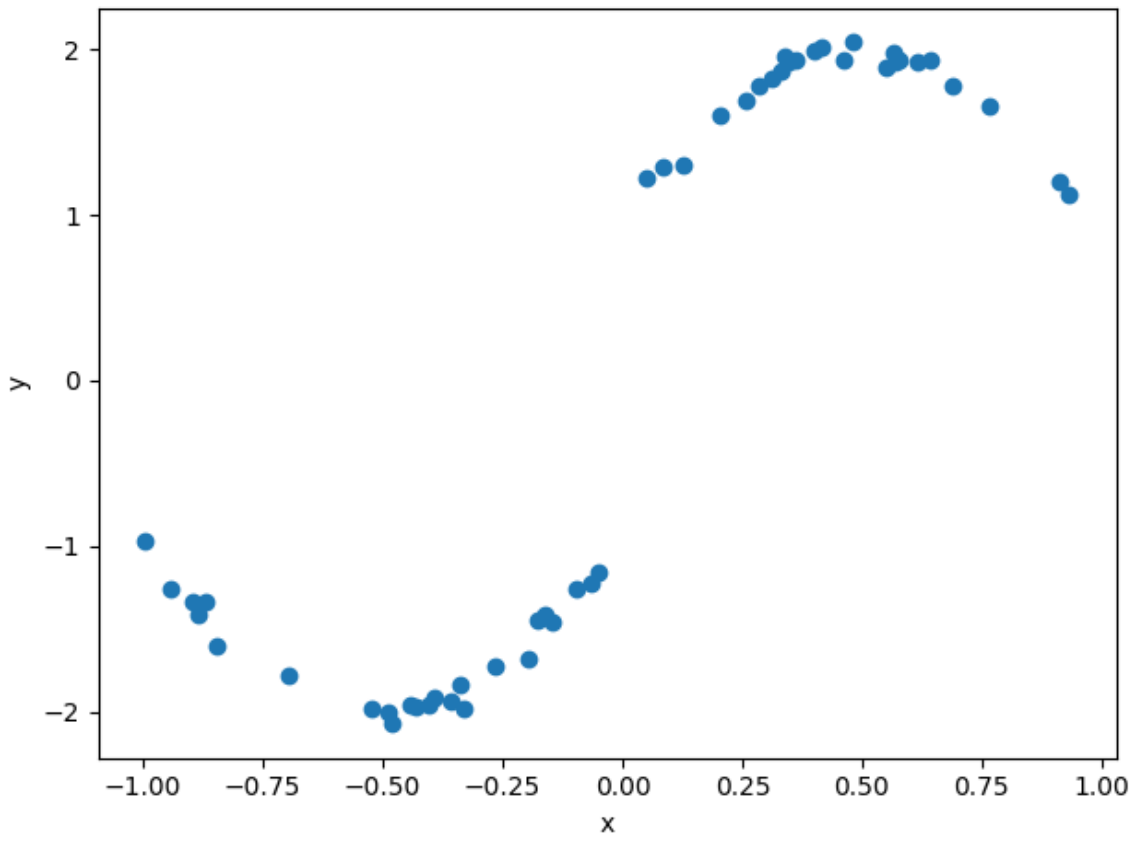
In [3…

```python
# Generate synthetic data
num_samples = 50
X = 2 * torch.rand(num_samples, 1) - 1
Y = torch.sin(torch.pi * X) + torch.sign(X) + 0.05 * torch.randn(X.shape)
```

A scatter plot shows the two regimes induced by the sign term. It is clear that a single smooth model is insufficient to capture this data set.

In [4…

```python
# Plot synthetic data
_, ax = plt.subplots()
ax.scatter(X, Y)
ax.set_xlabel("x")
ax.set_ylabel("y")
plt.tight_layout()
```



We instantiate two cubic `PolynomialChaosExpansion` experts and a `tanh`-activated `GatingNetwork` with width 16 and depth 1. Printing `num_params` reveals how many stochastic coefficients the joint model contains.

In [5…

```
# Create a surrogate model
width = 16
depth = 1
surrogate_model = MixtureOfExperts(
    experts=[
        PolynomialChaosExpansion(num_inputs=1, num_outputs=1, degree=3),
        PolynomialChaosExpansion(num_inputs=1, num_outputs=1, degree=3),
    ],
    gating_network=GatingNetwork(num_inputs=1, num_experts=2, width=width, depth=depth,
activation=torch.nn.functional.tanh)
)

num_params = surrogate_model.num_params()
print(f"This model has {num_params} parameters")
```

```
This model has 74 parameters
```

Each chaos expert has a split between stochastic (`IIDGaussianGroup`) and deterministic coefficients, while the gating network layers are grouped so their weights and biases can either remain deterministic or follow Gaussian priors. This structure controls which parts of the MoE carry epistemic uncertainty.

In [6…

```
# Create parameter groups
parameter_groups = [  # Experts
    IIDGaussianGroup("expert1_c0", 2),
    DeterministicGroup("expert1_c", 2),
    IIDGaussianGroup("expert2_c0", 2),
    DeterministicGroup("expert2_c", 2),
]
# Gating network
parameter_groups += [  # Input layer
    DeterministicGroup("gating_input_layer_weights", width),
    DeterministicGroup("gating_input_layer_biases", width),
]
for j in range(depth - 1):
    parameter_groups += [  # Hidden layers
        DeterministicGroup(f"gating_layer{j + 1}_weights", width * width),
        DeterministicGroup(f"gating_layer{j + 1}_biases", width),
    ]
parameter_groups += [  # Output layer
    IIDGaussianGroup("gating_output_weights", 2 * width),
    IIDGaussianGroup("gating_output_biases", 2),
]
```

`StochasticModel` fuses the surrogate with the parameter priors so we can sample expert/gate parameters jointly and compute their distribution regularizer during training.

In [7…

```
# Create stochastic model
model = StochasticModel(
    surrogate_model=surrogate_model, parameter_groups=parameter_groups
)
print(f"Created stochastic model with {model.num_params()} parameters")
```

```
Created stochastic model with 74 parameters
```

Specify training hyperparameters.

In [8…

```
# Training options
lr = 1e-2              # Learning rate
weight_decay = 1e-4    # Weight decay for AdamW
weight_factor = 1e-1   # Weight factor for distribution loss
num_epochs = 10000     # Number of epochs
num_samples = 100      # Number of parameter samples per epoch

# Learnable global precision
log_tau = torch.nn.Parameter(torch.tensor(0.0))  # τ = exp(log_tau)

# Optimizer: AdamW
optimizer = torch.optim.AdamW(model.parameters(), lr=lr, weight_decay=weight_decay)

# Loss
loss_fn = torch.nn.MSELoss(reduction="sum")

# Scheduler: OneCycleLR
scheduler = torch.optim.lr_scheduler.OneCycleLR(
    optimizer,
    max_lr=10 * lr,
    total_steps=num_epochs
)
```

Each epoch draws 100 parameter samples, evaluates the MoE on the training inputs, evaluates the data loss and adds the distribution regularizer weighted by `weight_factor`, and performs the optimizer and scheduler steps.

In [9…

```
# Train the stochastic model
for epoch in range(num_epochs):
    optimizer.zero_grad()

    # Evaluate parameters and model
    params = model.sample_parameters(num_samples=100)
    Y_hat = surrogate_model(X, params)

    # Losses
    data_loss = loss_fn(Y_hat, Y.unsqueeze(0).expand_as(Y_hat)) / num_samples
    distribution_loss = model.distribution_loss()
    total_loss = data_loss + weight_factor * distribution_loss

    # Backprop + step
    total_loss.backward()
    optimizer.step()
    scheduler.step()

    # Logging
    if (epoch + 1) % 1000 == 0:
        current_lr = scheduler.get_last_lr()[0]
        print(
            f"Epoch {epoch + 1:5d} | "
            f"learning rate = {current_lr:.6f} | "
            f"data loss = {data_loss.item():.4f} | "
            f"distribution loss = {distribution_loss.item():.4f} | "
            f"total loss = {total_loss.item():.4f}"
        )
```

```
Epoch  1000 | learning rate = 0.028015 | data loss = 8.0579 |
distribution loss = 69.4361 | total loss = 15.0015
Epoch  2000 | learning rate = 0.076029 | data loss = 6.9081 |
distribution loss = 55.0426 | total loss = 12.4124
Epoch  3000 | learning rate = 0.100000 | data loss = 1.0801 |
distribution loss = 62.7656 | total loss = 7.3567
Epoch  4000 | learning rate = 0.095039 | data loss = 1.0363 |
distribution loss = 62.4390 | total loss = 7.2802
Epoch  5000 | learning rate = 0.081157 | data loss = 1.1154 |
distribution loss = 62.2607 | total loss = 7.3415
Epoch  6000 | learning rate = 0.061104 | data loss = 1.0450 |
distribution loss = 62.3786 | total loss = 7.2829
Epoch  7000 | learning rate = 0.038852 | data loss = 1.1119 |
distribution loss = 62.2398 | total loss = 7.3358
Epoch  8000 | learning rate = 0.018808 | data loss = 1.0815 |
distribution loss = 62.1805 | total loss = 7.2996
Epoch  9000 | learning rate = 0.004942 | data loss = 1.0852 |
distribution loss = 62.2581 | total loss = 7.3110
Epoch 10000 | learning rate = 0.000000 | data loss = 1.0999 |
distribution loss = 62.1258 | total loss = 7.3125
```

With the model in eval mode we create 100 evenly spaced test inputs and sample 1000 stochastic forward passes to estimate the predictive distribution of the MoE across the interval.

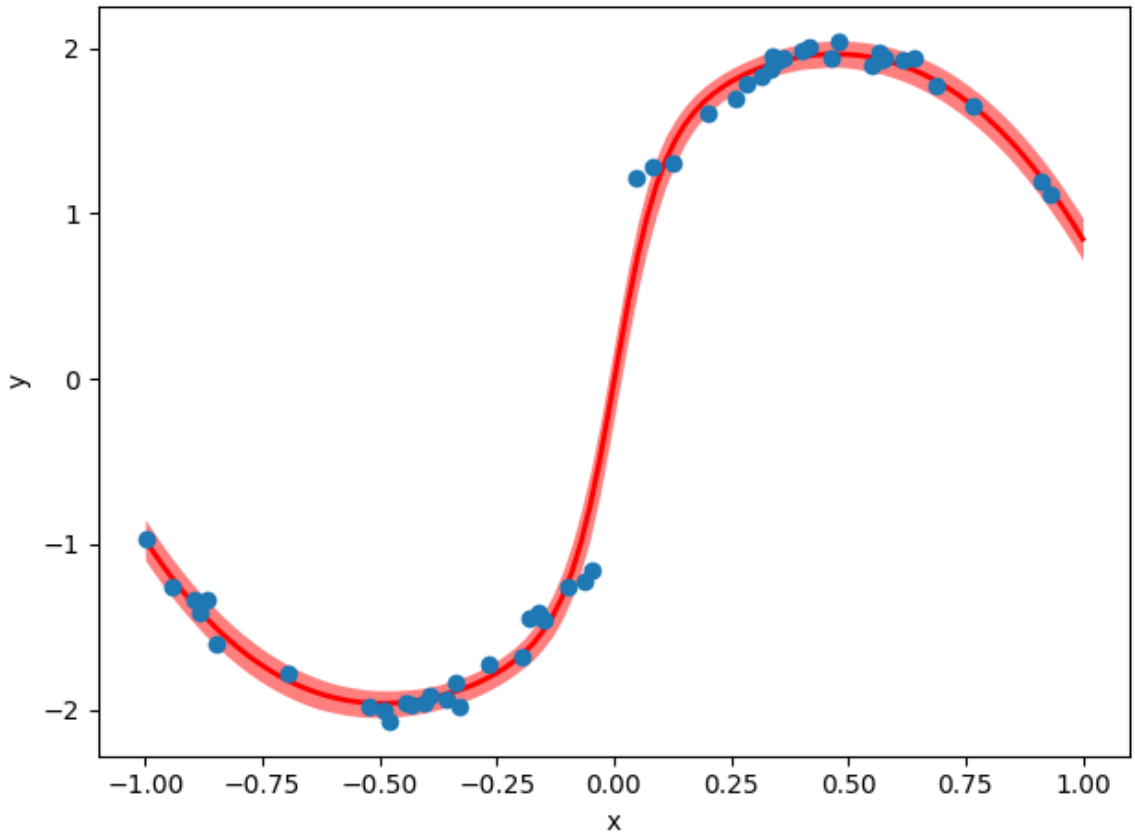
In [1…

```
# Evaluate the model
model.eval()
with torch.no_grad():
    X_test = torch.linspace(-1, 1, 100).unsqueeze(1)
    Y_test = model(X_test, num_samples=1000)
```

Quantiles of the sampled outputs form a shaded credible band and the sample mean gives the red curve, which we overlay with the raw training data to judge fit and uncertainty.

In [1…

```
# Plot prediction
_, ax = plt.subplots()
Q_test = torch.quantile(Y_test, torch.tensor([0.1, 0.9]), axis=0)
X_plot = X_test.squeeze(-1)
Q_plot = Q_test.squeeze(-1)
ax.fill_between(X_plot, Q_plot[0], Q_plot[-1], color="red", alpha=0.5, linewidth=0)
M_plot = Y_test.mean(axis=0).squeeze(-1)
ax.plot(X_plot, M_plot, color="red", linewidth=2)
ax.scatter(X, Y, zorder=99)
ax.set_xlabel("x")
ax.set_ylabel("y")
plt.tight_layout()
```



Using `get_gating_weights` we directly query the probabilistic gating network on the test grid, which yields the random mixing coefficients $ \pi_m(x)$ for each expert.

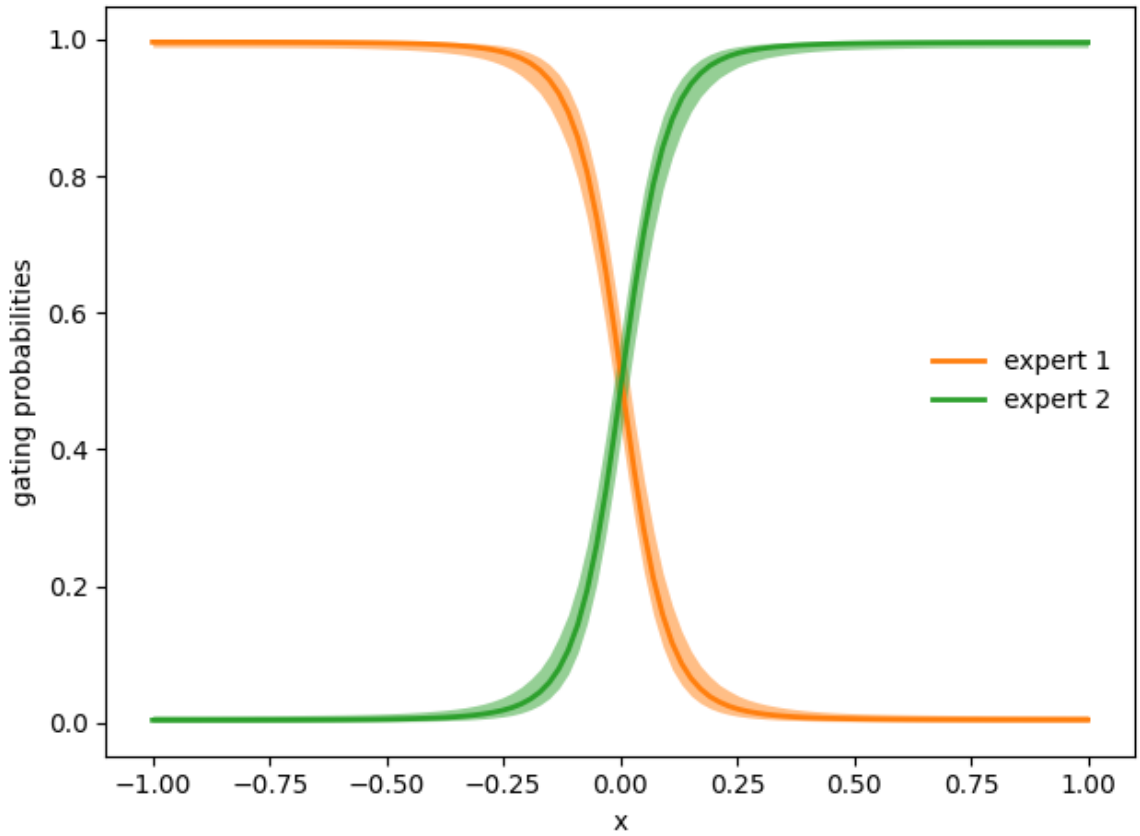
In [1…

```
# Evaluate the gating probabilities
with torch.no_grad():
    params = model.sample_parameters(1000)
    gating_weights = surrogate_model.get_gating_weights(X_test, params)
```

We plot the median and 80% interval of the gating probabilities so it is clear how responsibility shifts between experts as x crosses zero.

In [1…

```
# Plot gating probabilities
_, ax = plt.subplots()
for expert in range(2):
    Q_test = torch.quantile(gating_weights[:, :, expert], torch.tensor([0.1, 0.5, 0.9]),
axis=0)
    X_plot = X_test.squeeze(-1)
    Q_plot = Q_test.squeeze(-1)
    # ax.plot(gating_weights[:, :, expert])
    ax.fill_between(X_plot, Q_plot[0], Q_plot[-1], color=f"C{expert + 1}", alpha=0.5,
linewidth=0)
    ax.plot(X_plot, Q_plot[1], color=f"C{expert + 1}", linewidth=2, label=f"expert
{expert + 1}")
ax.set_xlabel("x")
ax.set_ylabel("gating probabilities")
ax.legend(frameon=False)
plt.tight_layout()
```



`get_expert_outputs` evaluates each expert separately (still under parameter uncertainty) so we can inspect what functions the gate is blending together.

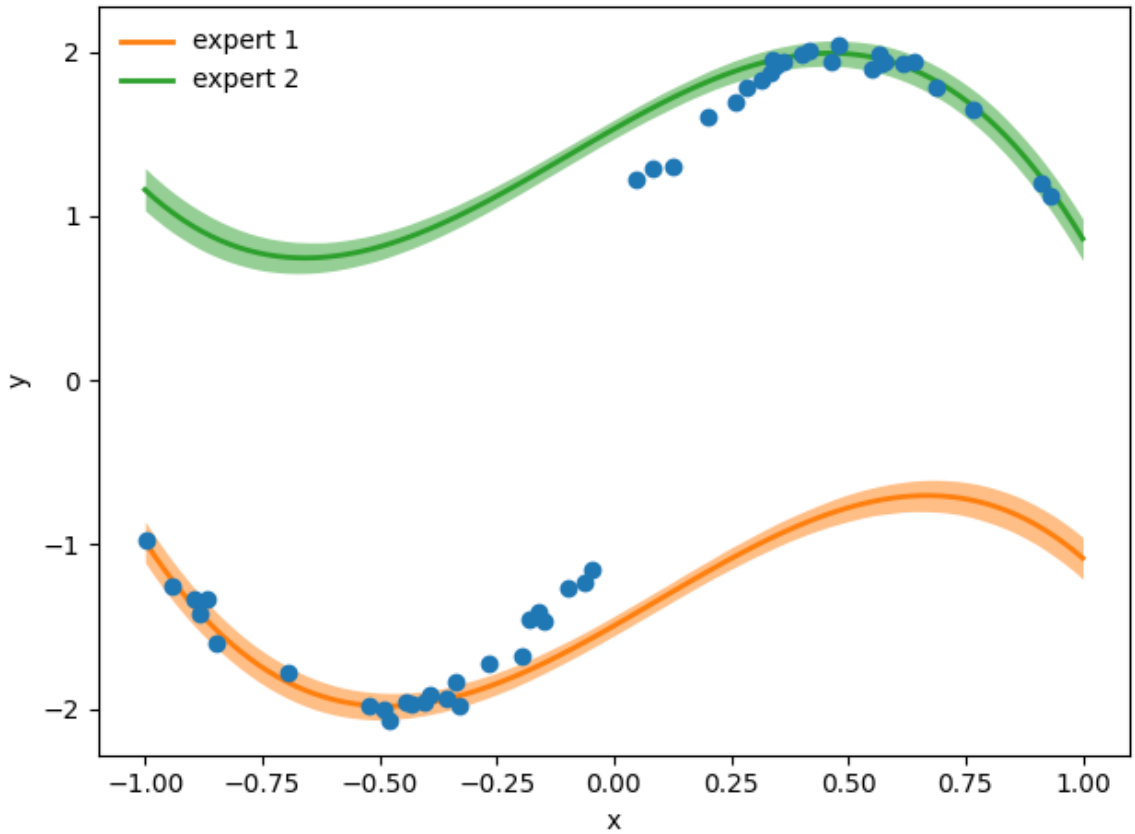
In [1…

```
# Evaluate the expert outputs
with torch.no_grad():
    params = model.sample_parameters(1000)
    expert_outputs = surrogate_model.get_expert_outputs(X_test, params)
```

Filling between quantiles of each expert's predictions, along with their means and the observed data, shows how the experts divide the regression task and how their uncertainty compares.

In [1…

```
# Plot expert outputs
_, ax = plt.subplots()
ax.scatter(X, Y, zorder=99)
for expert in range(2):
    Q_test = torch.quantile(expert_outputs[:, expert, :, :], torch.tensor([0.1, 0.5,
0.9]), axis=0)
    X_plot = X_test.squeeze(-1)
    Q_plot = Q_test.squeeze(-1)
    ax.fill_between(X_plot, Q_plot[0], Q_plot[-1], color=f"C{expert + 1}", alpha=0.5,
linewidth=0)
    M_plot = expert_outputs[:, expert, :, :].mean(axis=0).squeeze(-1)
    ax.plot(X_plot, M_plot, color=f"C{expert + 1}", linewidth=2, label=f"expert {expert
+ 1}")
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.legend(frameon=False)
plt.tight_layout()
```



In […

4.4 Full-Covariance Polynomial Chaos

We revisit the wavy one-dimensional regression toy problem and let `pypolymix` infer a joint Gaussian over every polynomial chaos coefficient. The notebook mirrors `nn.ipynb` and `moe.ipynb` by stepping through data generation, surrogate construction, stochastic training, and posterior diagnostics with short explanations along the way.

In [1…

```
# Import statements
import matplotlib.pyplot as plt
import torch

from pypolymix.parameter_groups import GaussianGroup
from pypolymix.surrogate_models import PolynomialChaosExpansion
from pypolymix import StochasticModel
```

In [2…

```
# Set random seed
_ = torch.manual_seed(1234)
```

In [3…

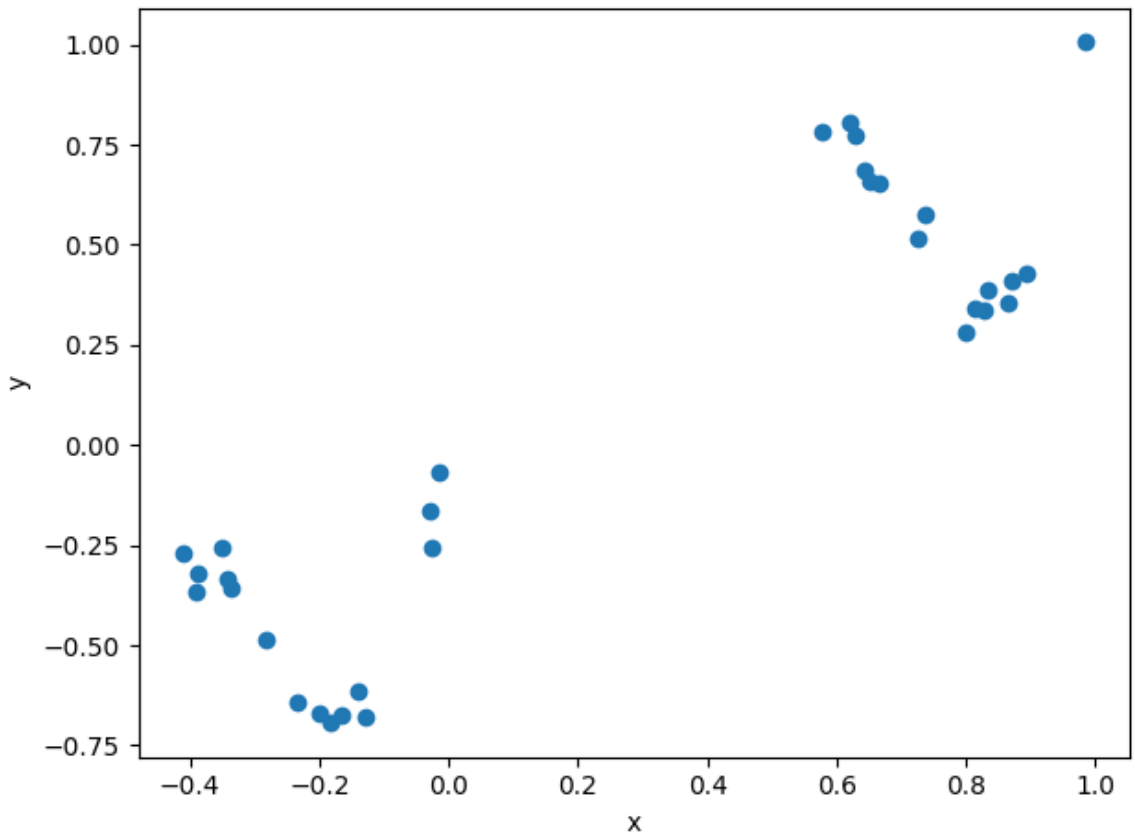
```
# Generate synthetic data
n_train = 32
noise = 0.02

x_train = torch.rand(n_train, 1)
x_train[:n_train // 2] *= -1
x_train[n_train // 2:] += 1
x_train /= 2

eps1 = noise * torch.randn(n_train, 1)
eps2 = noise * torch.randn(n_train, 1)
eps3 = noise * torch.randn(n_train, 1)

y_train = x_train + 0.3 * torch.sin(2. * torch.pi * (x_train + eps1)) + 0.3 *
torch.sin(4. * torch.pi * (x_train + eps2)) + eps3
```

We sample `n_train = 32` inputs split evenly across the negative and positive halves of the domain, blend two sinusoids with a linear drift, and add small Gaussian noise.

A quick scatter plot sets expectations for the frequency content and the observation noise that the surrogate must ultimately match.

In [4…

```
# Plot synthetic data
_, ax = plt.subplots()
ax.scatter(x_train, y_train)
ax.set_xlabel("x")
ax.set_ylabel("y")
plt.tight_layout()
```



A degree-7 `PolynomialChaosExpansion` is flexible enough to follow the multi-frequency signal while staying cheap to evaluate, so thousands of stochastic forward passes remain affordable.

In [5…

```
# Create a surrogate model
degree = 7
surrogate_model = PolynomialChaosExpansion(num_inputs=1, num_outputs=1, degree=degree)
num_params = surrogate_model.num_params()
print(f"This model has {num_params} parameters")
```

```
This model has 8 parameters
```

`GaussianGroup` wraps the full coefficient vector so `pypolymix` learns a dense mean vector and covariance matrix. Allowing the weights to correlate is important because adjacent polynomial bases tend to move together once conditioned on data.

In [6…

```
# Create parameter groups
parameters = GaussianGroup("params", surrogate_model.num_params())
```

`StochasticModel` stitches the deterministic surrogate and parameter group together, handles coefficient sampling, and exposes helpers such as `distribution_loss()` for regularising the posterior.

In [7…

```
# Create stochastic model
model = StochasticModel(surrogate_model, parameters)
print(f"Created stochastic model with {model.num_params()} parameters")
```

```
Created stochastic model with 8 parameters
```

We mirror the other tutorials by collecting the standard hyperparameters (`num_samples`, `num_epochs`, `lr`, `weight_decay`, `weight_factor`), introducing a learnable `log_tau` so the data term becomes $\tau \cdot \text{MSE}$, and driving AdamW with a `OneCycleLR` schedule.

In [8…

```
# Training options
lr = 1e-3              # Learning rate
weight_decay = 1e-4    # Weight decay for AdamW
weight_factor = 5e-3   # Weight factor for distribution loss
num_epochs = 10000     # Number of epochs
num_samples = 100      # Number of parameter samples per epoch

# Optimizer: AdamW
optimizer = torch.optim.AdamW(model.parameters(), lr=lr, weight_decay=weight_decay)

# Loss
loss_fn = torch.nn.MSELoss(reduction="sum")

# Scheduler: OneCycleLR
scheduler = torch.optim.lr_scheduler.OneCycleLR(
    optimizer,
    max_lr=10 * lr,
    total_steps=num_epochs
)
```

Each epoch draws `num_samples` coefficient realizations, evaluates the surrogate on all training inputs, aligns the target tensor so that `torch.nn.MSELoss` can operate in batch, and mixes the resulting data term with the distribution regularizer.

We keep a short list of metrics and let `tabulate` render the epoch, learning rate, and loss components in a compact table (falling back to the plain text formatter if `tabulate` is unavailable).

In [9…

```
# Train the stochastic model
for epoch in range(num_epochs):
    optimizer.zero_grad()

    # Draw posterior samples and evaluate the surrogate on the training set
    params = model.sample_parameters(num_samples=num_samples)
    predictions = surrogate_model(x_train, params)

    # Compute losses
    target = y_train.unsqueeze(0).expand_as(predictions)
    data_loss = loss_fn(predictions, target) / num_samples
    distribution_loss = model.distribution_loss()
    total_loss = data_loss + weight_factor * distribution_loss

    # Backpropagation + optimizer + scheduler steps
    total_loss.backward()
    optimizer.step()
    scheduler.step()

    # Logging
    if (epoch + 1) % 1000 == 0:
        current_lr = scheduler.get_last_lr()[0]
        print(
            f"Epoch {epoch + 1:5d} | "
            f"learning rate = {current_lr:.6f} | "
            f"data loss = {data_loss.item():.4f} | "
            f"distribution loss = {distribution_loss.item():.4f} | "
            f"total loss = {total_loss.item():.4f}"
        )
```

```
Epoch  1000 | learning rate = 0.002801 | data loss = 0.2711 |
distribution loss = 19.0272 | total loss = 0.3663
Epoch  2000 | learning rate = 0.007603 | data loss = 0.1933 |
distribution loss = 21.1081 | total loss = 0.2988
Epoch  3000 | learning rate = 0.010000 | data loss = 0.1932 |
distribution loss = 20.8932 | total loss = 0.2977
Epoch  4000 | learning rate = 0.009504 | data loss = 0.1917 |
distribution loss = 20.8753 | total loss = 0.2961
Epoch  5000 | learning rate = 0.008116 | data loss = 0.1925 |
distribution loss = 20.7089 | total loss = 0.2961
Epoch  6000 | learning rate = 0.006110 | data loss = 0.1911 |
distribution loss = 20.7517 | total loss = 0.2948
Epoch  7000 | learning rate = 0.003885 | data loss = 0.1921 |
distribution loss = 21.4562 | total loss = 0.2994
Epoch  8000 | learning rate = 0.001881 | data loss = 0.1906 |
distribution loss = 20.6770 | total loss = 0.2940
Epoch  9000 | learning rate = 0.000494 | data loss = 0.1915 |
distribution loss = 20.7908 | total loss = 0.2955
Epoch 10000 | learning rate = 0.000000 | data loss = 0.1905 |
distribution loss = 20.8625 | total loss = 0.2948
```

After training we put the model in `eval()` mode, create a dense grid of test inputs, and draw stochastic forward passes inside `torch.no_grad()` so the predictive distribution over the domain can be summarized.

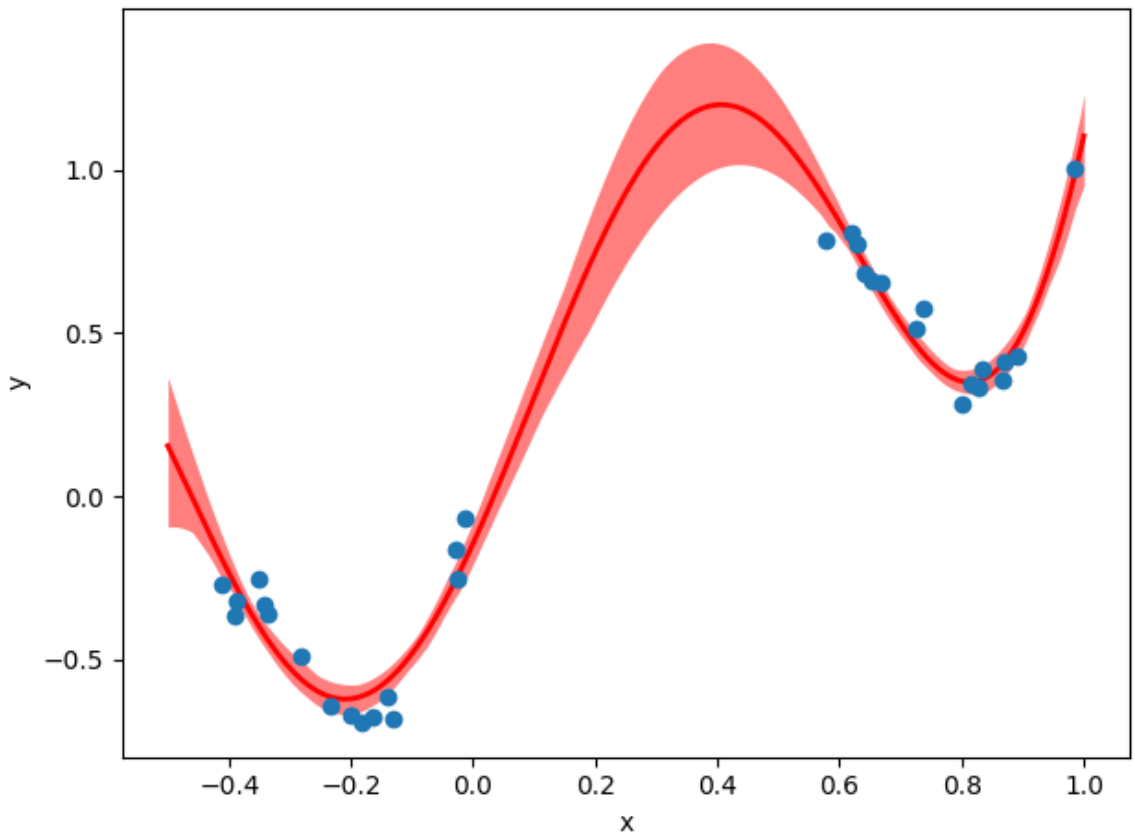
In [1…

```python
# Evaluate the model
model.eval()
with torch.no_grad():
    x_test = torch.linspace(-0.5, 1, 1001).unsqueeze(1)
    y_test = model(x_test, num_samples=100)
```

We summarize the sampled predictions by plotting the sample mean and the central 98% interval (1%-99% quantiles), then overlay the noisy observations to visually assess how well the stochastic PCE tracks the signal.

In [1…

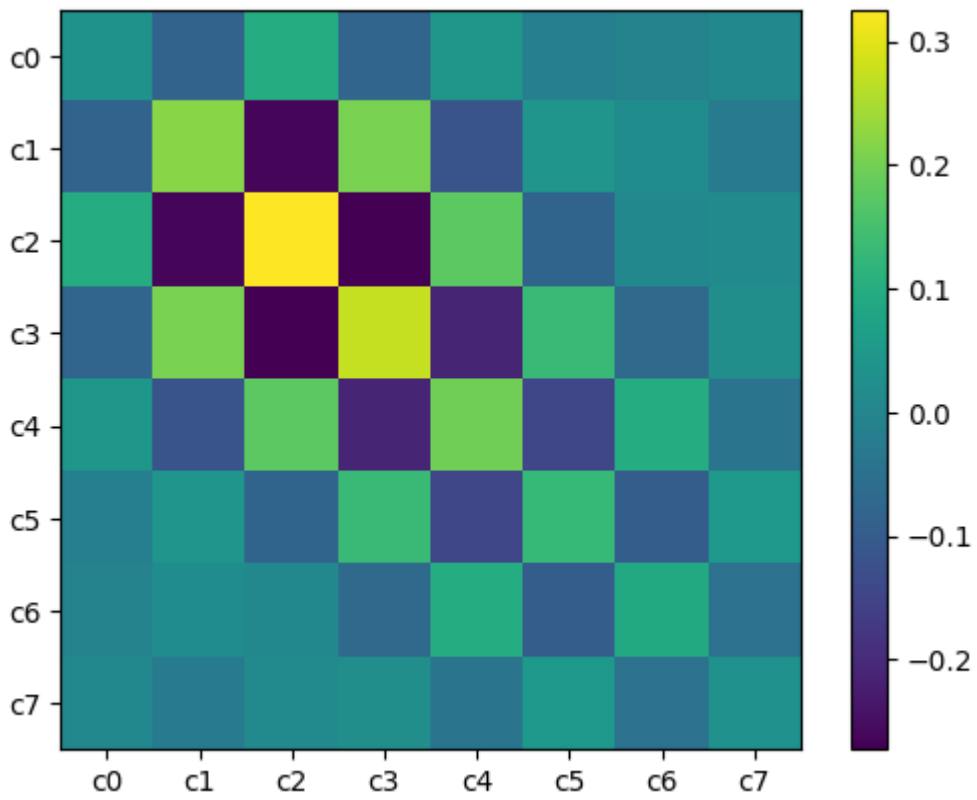
```python
# Plot prediction
_, ax = plt.subplots()
Q_test = torch.quantile(y_test, torch.tensor([0.01, 0.99]), axis=0)
X_plot = x_test.squeeze(-1)
Q_plot = Q_test.squeeze(-1)
ax.fill_between(X_plot, Q_plot[0], Q_plot[-1], color="red", alpha=0.5, linewidth=0)
M_plot = y_test.mean(axis=0).squeeze(-1)
ax.plot(X_plot, M_plot, color="red", linewidth=2)
ax.scatter(x_train, y_train, zorder=99)
ax.set_xlabel("x")
ax.set_ylabel("y")
plt.tight_layout()
```



Finally we inspect the posterior covariance between polynomial chaos coefficients to understand how uncertainty couples the basis functions. A heatmap highlights dominant directions or nearly deterministic weights.

In [1…

```
# Inspect the posterior covariance
with torch.no_grad():
    gaussian_group = model.parameter_groups[0]
    posterior = gaussian_group.variational_distribution()
    covariance = posterior.covariance_matrix
    samples = gaussian_group.sample_parameters(num_samples=2048)
    correlation = torch.corrcoef(samples.T)

# Plot correlation matrix
plt.imshow(covariance)
plt.colorbar()

num_params = surrogate_model.num_params()
plt.xticks(range(num_params), [f"c{i}" for i in range(num_params)])
plt.yticks(range(num_params), [f"c{i}" for i in range(num_params)])

plt.show()
```



In […

4.5 Priors Guide Ambiguous Line Fits

This example illustrates how informative priors resolve ambiguity when the data alone cannot identify a low-order model. We fit a 1D polynomial chaos surrogate (a line) to a tiny set of observations that cover only a narrow portion of the domain. Two competing priors (one favoring positive slopes, the other negative) lead to very different extrapolations even though the data fit quality is similar.

In [1…

```
# Import statements
import matplotlib.pyplot as plt
import torch
import torch.nn.functional as F

from pypolymix import StochasticModel
from pypolymix.parameter_groups import GaussianGroup
from pypolymix.priors import GaussianPrior
from pypolymix.surrogate_models import PolynomialChaosExpansion
```

In [2…

```
# Set random seed
_ = torch.manual_seed(2025)
```

We collect only six noisy observations clustered near the origin. Within that window the response is almost flat, so many slope/intercept pairs explain the data equally well. Outside the observed region, however, the different lines diverge dramatically.

In [3…

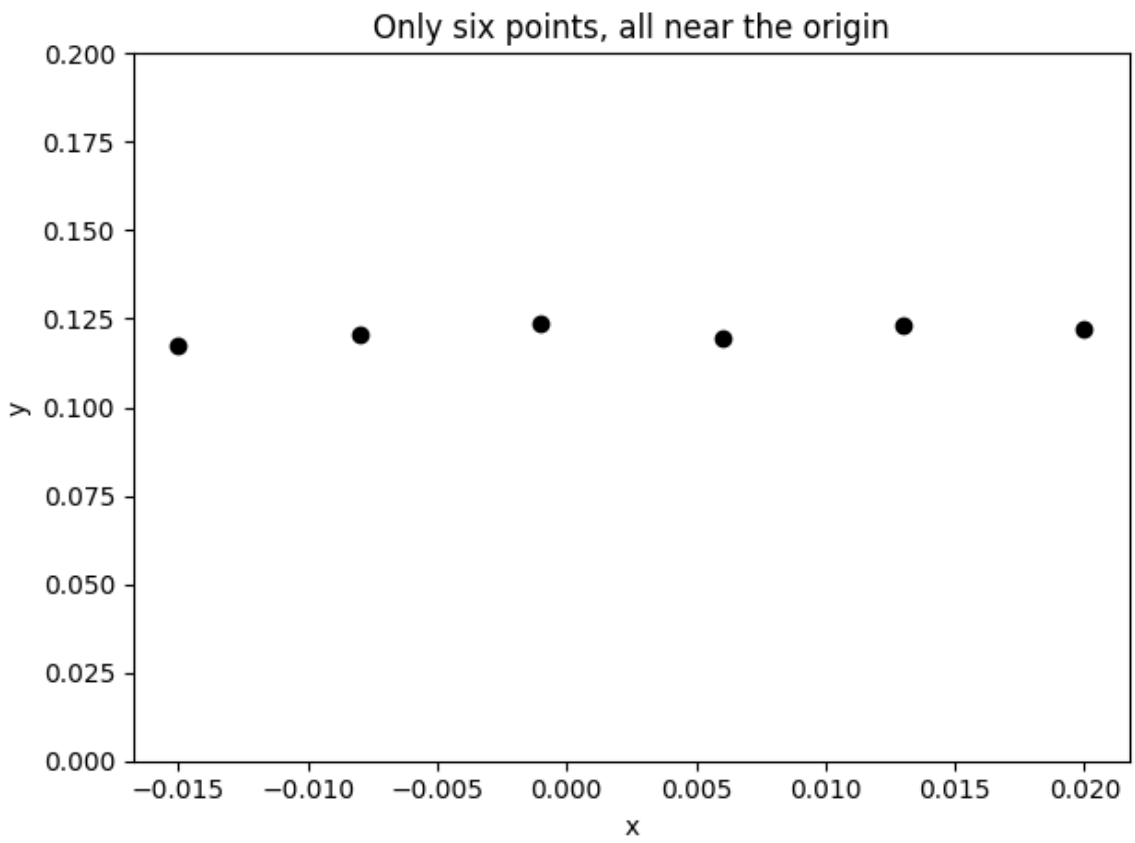
```
# Generate scarce, ambiguous training data
num_points = 6
x_train = torch.linspace(-0.015, 0.02, num_points).unsqueeze(-1)
true_intercept = 0.12
true_slope = 0.0
noise = 0.003 * torch.randn_like(x_train)
y_train = true_intercept + true_slope * x_train + noise

print(f"x range: {x_train.min().item():+.3f} to {x_train.max().item():+.3f}")
print(f"mean y: {y_train.mean().item():.3f}")
```

```
x range: -0.015 to +0.020
mean y: 0.121
```

With so little spread in x, visual inspection already hints that the slope is underdetermined.

In [4…

```
# Plot the scarce observations
_, ax = plt.subplots()
ax.scatter(x_train, y_train, color="black")
ax.set_xlabel("x")
ax.set_ylim(0, 0.2)
ax.set_ylabel("y")
ax.set_title("Only six points, all near the origin")
plt.tight_layout()
```



Next we define two priors over the line coefficients. Both favor an intercept of 0.12, which matches the observed measurements, but one insists on a positive slope (+1) and the other on an equally strong negative slope (-1). The precision of the slope prior is considerably higher than that of the intercept, so we expect it to dominate whenever the data leave the slope unconstrained.

In [5…

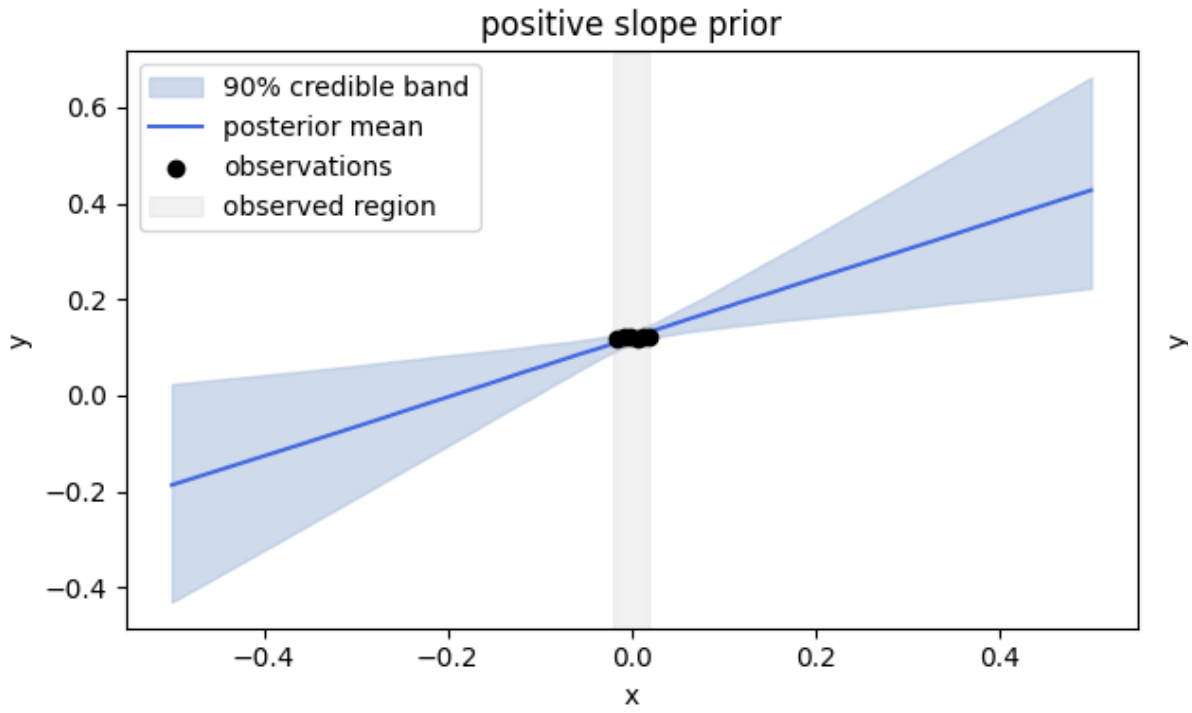
```python
# Build and train stochastic line models under different priors
def build_model(name: str, slope_prior: float) -> StochasticModel:
    surrogate = PolynomialChaosExpansion(num_inputs=1, num_outputs=1, degree=1)
    prior_mean = torch.tensor([0.12, slope_prior])
    prior_std = torch.tensor([0.04, 0.35])
    covariance = torch.diag(prior_std ** 2)
    prior = GaussianPrior(mean=prior_mean, covariance_matrix=covariance)
    group = GaussianGroup(name, surrogate.num_params(), prior=prior)
    return StochasticModel(surrogate, group)

training_options = dict(
    num_epochs=4000,
    lr=3e-3,
    weight_decay=1e-4,
    weight_factor=5e-5,
    num_parameter_samples=128,
)

priors = {
    "positive slope prior": 1,
    "negative slope prior": -1,
}

models = {}
for label, slope_mean in priors.items():
    model = build_model(label, slope_mean)
    optimizer = torch.optim.AdamW(
        model.parameters(),
        lr=training_options["lr"],
        weight_decay=training_options["weight_decay"],
    )

    print(f"Training model with {label} ...")
    for epoch in range(training_options["num_epochs"]):
        optimizer.zero_grad()
        predictions = model(x_train,
num_samples=training_options["num_parameter_samples"])
        target = y_train.unsqueeze(0).expand_as(predictions)

        data_loss = F.mse_loss(predictions, target)
        distribution_loss = model.distribution_loss()
        total_loss = data_loss + training_options["weight_factor"] * distribution_loss

        total_loss.backward()
        optimizer.step()

        if (epoch + 1) % 1000 == 0:
            print(
                f"Epoch {epoch + 1:4d}: data_loss={data_loss.item():.6f} | "
                f"distribution_loss={distribution_loss.item():.6f}"
            )

    models[label] = model
```

```
Training model with positive slope prior ...
Epoch 1000: data_loss=0.000682 | distribution_loss=0.898466
Epoch 2000: data_loss=0.000295 | distribution_loss=1.189126
Epoch 3000: data_loss=0.000149 | distribution_loss=1.479722
Epoch 4000: data_loss=0.000133 | distribution_loss=1.723111
Training model with negative slope prior ...
Epoch 1000: data_loss=0.000729 | distribution_loss=1.219246
Epoch 2000: data_loss=0.000248 | distribution_loss=1.528454
Epoch 3000: data_loss=0.000170 | distribution_loss=1.828431
Epoch 4000: data_loss=0.000136 | distribution_loss=2.057850
```

The data loss plateaus at similar values for both models, yet their priors push one solution to slope upward and the other to slope downward. We now visualize how that prior knowledge changes extrapolations away from the observed data.

In [6…

```python
# Compare predictive distributions across the domain
x_test = torch.linspace(-0.5, 0.5, 400).unsqueeze(-1)
quantile_levels = torch.tensor([0.05, 0.5, 0.95])

with torch.no_grad():
    fig, axes = plt.subplots(1, 2, figsize=(12, 4), sharey=True)
    for ax, (label, model) in zip(axes, models.items()):
        samples = model(x_test, num_samples=512).squeeze(-1)
        quantiles = torch.quantile(samples, quantile_levels, dim=0)

        ax.fill_between(
            x_test.squeeze(-1).numpy(),
            quantiles[0].numpy(),
            quantiles[2].numpy(),
            color="lightsteelblue",
            alpha=0.6,
            label="90% credible band",
        )
        ax.plot(x_test, quantiles[1], color="royalblue", label="posterior mean")
        ax.scatter(x_train, y_train, color="black", zorder=3, label="observations")
        ax.axvspan(-0.02, 0.02, color="lightgray", alpha=0.3, label="observed region")
        ax.set_title(label)
        ax.set_xlabel("x")
        ax.set_ylabel("y")
        ax.legend(loc="best")

    plt.tight_layout()
```



The priors dominate outside the tiny gray band that marks where we have data. Because the likelihood provides almost no slope information, the positive-slope prior extrapolates upward and the negative-slope prior extrapolates downward, perfectly illustrating how Bayesian priors resolve non-identifiability.

In [7…

```
# Inspect posterior parameter summaries
for label, model in models.items():
    gaussian_group = model.parameter_groups[0]
    posterior = gaussian_group.variational_distribution()
    mean = posterior.mean
    std = posterior.stddev
    print(
        f"{label:>24s} -> intercept={mean[0]:+.3f} ± {std[0]:.3f}, "
        f"slope={mean[1]:+.3f} ± {std[1]:.3f}"
    )
```

```
    positive slope prior -> intercept=+0.121 ± 0.008, slope=+0.640 ± 0.274
    negative slope prior -> intercept=+0.122 ± 0.008, slope=-0.542 ± 0.275
```

Both solutions honor the six data points but lock onto different slope priors. Whenever measurements are scarce or localized, deliberately chosen priors allow us to encode domain knowledge and pick the physically plausible branch of the solution.

5. Acknowledgements

5.1 Acknowledgments

This work was supported by the U.S. Department of Energy, Office of Nuclear Energy, and Office of Science, Office of Advanced Scientific Computing Research through the Scientific Discovery through through the FASTMath Institute and the Advanced Computing project on Simulation of the Response of Structural Metals in Molten Salt Environment.

This software library has been authored by employees of National Technology and Engineering Solutions of Sandia, LLC under Contract No. DE-NA0003525 with the U.S. Department of Energy (DOE).